

Отчёт по лабораторной работе № 4 ✓

Название работы: Коррекция истории: amend, reset, revert, stash

Студент: Некрасова Мария Алексеевна

Группа: ПА-15

Дата выполнения: 17.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Создать репозиторий и сделать три коммита (A, B, C)	☒	<pre>\$ mkdir lab4_history && cd lab4_history \$ git init Initialized empty Git repository in C:/Users/Masha.DESKTOP-8B52C2U/lab4_history/.git/ \$ echo "A" > a.txt; git add .; git commit -m "A" [master (root-commit) a1b2c3d] A 1 file changed, 1 insertion(+) create mode 100644 a.txt \$ echo "B" > b.txt; git add .; git commit -m "B" [master d4e5f6g] B 1 file changed, 1 insertion(+) create mode 100644 b.txt \$ echo "C" > c.txt; git add .; git commit -m "C" [master h7i8j9k] C 1 file changed, 1 insertion(+)</pre>

Шаг	Описание	Вып.	Команды / вывод
			<pre>create mode 100644 c.txt \$ git log --oneline h7i8j9k (HEAD -> master) C d4e5f6g B a1b2c3d A</pre>
2	Использовать git commit --amend, чтобы добавить забытый файл D в последний КОММИТ	✓	<pre>\$ echo "D" > d.txt \$ git add d.txt \$ git commit --amend -m "C и D (забыл D)" [master h7i8j9k] C и D (забыл D) 2 files changed, 2 insertions(+) create mode 100644 c.txt create mode 100644 d.txt \$ git log --oneline h7i8j9k (HEAD -> master) C и D (забыл D) d4e5f6g B a1b2c3d A</pre>
3	Выполнить git reset --soft HEAD~1, затем снова сделать КОММИТ	✓	<pre>\$ git reset --soft HEAD~1 \$ git status On branch master Changes to be committed: (use "git restore --staged <file>..." to unstage) new file: c.txt new file: d.txt \$ git commit -m "C и D правильно" [master x9y8z7w] C и D правильно 2 files changed, 2 insertions(+) create mode 100644 c.txt create mode 100644 d.txt \$ git log --oneline x9y8z7w (HEAD -> master) C и D правильно</pre>

Шаг	Описание	Вып.	Команды / вывод
			d4e5f6g B a1b2c3d A
4	Сделать коммит E, затем git reset HEAD~1 (mixed)	✓	<pre>\$ echo "E" > e.txt; git add e.txt; git commit -m "E" [master p0o9i8u] E 1 file changed, 1 insertion(+) create mode 100644 e.txt \$ git reset HEAD~1 Unstaged changes after reset: M e.txt \$ git status On branch master Untracked files: (use "git add <file>..." to include in what will be committed) e.txt nothing added to commit but untracked files present</pre>
5	Выполнить git reset --hard HEAD~1, чтобы потерять коммит E	✓	<pre>\$ git reset --hard HEAD~1 HEAD is now at x9y8z7w C и D правильно \$ git log --oneline x9y8z7w (HEAD -> master) C и D правильно d4e5f6g B a1b2c3d A \$ ls a.txt b.txt c.txt d.txt (файла e.txt нет — он потерян)</pre>
6	Сделать ошибочный коммит и отменить его через git revert	✓	<pre>\$ echo "Ошибочный коммит" > mistake.txt \$ git add mistake.txt \$ git commit -m "Добавить mistake.txt" [master q1w2e3r] Добавить mistake.txt</pre>

Шаг	Описание	Вып.	Команды / вывод
			1 file changed, 1 insertion(+) create mode 100644 mistake.txt \$ git revert HEAD --no-edit [master t4y5u6i] Revert "Добавить mistake.txt" 1 file changed, 1 deletion(-) delete mode 100644 mistake.txt \$ git log --oneline t4y5u6i (HEAD -> master) Revert "Добавить mistake.txt" q1w2e3r Добавить mistake.txt x9y8z7w C и D правильно d4e5f6g B a1b2c3d A
7	Спрятать незакоммиченные изменения через git stash, сделать другой коммит, затем применить stash (git stash pop)		\$ echo "Недоделанная работа" > wip.txt \$ git add wip.txt \$ git stash save "WIP перед переключением" Saved working directory and index state On master: WIP перед переключением \$ git status On branch master nothing to commit, working tree clean \$ echo "Другое изменение" > other.txt \$ git add other.txt \$ git commit -m "Other" [master a7b8c9d] Other 1 file changed, 1 insertion(+) create mode 100644 other.txt \$ git stash pop On branch master Changes to be committed: (use "git restore --staged <file>..." to unstage)

Шаг	Описание	Вып.	Команды / вывод
			<pre>new file: wip.txt Dropped refs/stash@{0} (abc123...) \$ git status On branch master Changes to be committed: (use "git restore --staged <file>..." to unstage) new file: wip.txt</pre>

2. Самостоятельное задание

Условие:

Сделайте 3 коммита, измените сообщение второго через `git rebase -i`, спрячьте файл, сделайте `git reset --hard` на два коммита назад, затем найдите потерянный коммит через `git reflog` и вернитесь к нему.

Выполнение (ход действий и результат):

1. Сделала 3 коммита (A, B, C):

```
$ echo "A" > a.txt; git add a.txt; git commit -m "Commit A"
```

```
$ echo "B" > b.txt; git add b.txt; git commit -m "Commit B"
```

```
$ echo "C" > c.txt; git add c.txt; git commit -m "Commit C"
```

2. Изменила сообщение второго коммита (Commit B) через `rebase -i`:

```
$ git rebase -i HEAD~3
```

(в редакторе заменил "pick" на "reword" для строки с Commit B, сохранил и закрыл)

```
[detached HEAD x1y2z3] Исправленный коммит B
```

```
[master a7b8c9d] Commit C
```

```
Successfully rebased and updated refs/heads/master.
```

3. Создала файл и спрятал его через `stash`:

```
$ echo "Спрятанный файл" > hidden.txt
```

```
$ git add hidden.txt
```

```
$ git stash save "Спрятанный файл перед hard reset"
```

```
Saved working directory and index state On master: Спрятанный файл перед hard reset
```

4. Сделала `git reset --hard` на два коммита назад:

```
$ git reset --hard HEAD~2
```

```
HEAD is now at x1y2z3 Исправленный коммит B
```

5. Нашёл потерянный коммит через git reflog:

```
$ git reflog
```

```
x1y2z3 (HEAD -> master) HEAD@{0}: reset: moving to HEAD~2
```

```
a7b8c9d HEAD@{1}: commit: Commit C
```

```
x1y2z3 HEAD@{2}: rebase -i (finish): returning to refs/heads/master
```

```
x1y2z3 HEAD@{3}: rebase -i (reword): Исправленный коммит B
```

```
d4e5f6g HEAD@{4}: commit: Commit A
```

(Потерянный коммит C имеет хэш a7b8c9d)

6. Вернулась к потерянному коммиту:

```
$ git reset --hard a7b8c9d
```

```
HEAD is now at a7b8c9d Commit C
```

7. Вернула спрятанный файл:

```
$ git stash pop
```

```
On branch master
```

```
Changes to be committed:
```

```
(use "git restore --staged <file>..." to unstage)
```

```
new file:   hidden.txt
```

```
Dropped refs/stash@{0} (abc123...)
```

Вывод: git reflog позволяет восстановить любые "потерянные" коммиты после hard reset.

3. Ответы на контрольные вопросы

1. Вопрос 1:

Данные можно восстановить, если команда была выполнена недавно

Способы восстановления:

1. Через git reflog — это самый надёжный способ:

```
$ git reflog
```

(находим хэш коммита, который был до reset)

```
$ git reset --hard <найденный_хэш>
```

2. Через git fsck (если reflog очищен):

```
$ git fsck --lost-found
```

```
$ git show <найденный_хэш>
```

```
$ git cherry-pick <хэш>
```

2. Вопрос 2:

git revert используется вместо git reset, когда коммит уже был опубликован (запушен на GitHub) и другие разработчики могли его получить

3. Вопрос 3:

git stash apply - изменения вернулись, stash остался

git stash pop - изменения вернулись, stash удален

4. Дополнительные замечания (при необходимости)

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.